EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Introduction to Pydantic

Instructor:
Arash Saifhashemi

Structure makes output usable by code.

ONE QUESTION

When is Assignment 1 due, and what should I submit?

FREE-FORM PROSE

“Assignment 1 is due around Week 3. Submit code, tests, a brief design explanation, and failed cases.”

People can infer the fields. Code must guess the boundaries.

model_output.json

```
{
  "assignment": 1,
  "due": "start of Week 3",
  "requirements": [
    "code", "tests",
    "design note", "failed cases"
  ],
  "source": "syllabus"
}
```

PARSER READS NAMED FIELDS

Predictable structure improves parsing. It does not guarantee factual correctness.

A Pydantic model is an executable contract.

1 Shape

Names and types define accepted input.

schema.py

```python
from pydantic import BaseModel, ConfigDict, Field

class AssignmentAnswer(BaseModel):
    model_config = ConfigDict(
        extra="forbid", strict=True
    )
    assignment: int = Field(ge=1, le=4)
    due: str
    requirements: list[str]
    source: str
```

2 Policy

Unknown keys and coercion are decided before validation.

3 Range

The integer must also fall within 1 through 4.

One class → JSON Schema for the model API → typed object or ValidationError in the host

Each setting blocks a different bad input.

MODEL-GENERATED INPUT	EXACT CONTRACT SPAN	WHY IT FAILS
`{"assignment":"1"}`	`strict=True`	No coercion: an integer must arrive as an integer.
`{"assignment":0}`	`Field(ge=1, le=4)`	The type is right, but zero is outside the supported range.
`{"assignment":1, "delete":true}`	`extra="forbid"`	Unknown keys are rejected instead of silently ignored.

Type, value range, and allowed fields are independent checks.

A Pydantic model emits JSON Schema.

`AssignmentAnswer.model_json_schema()`

AssignmentAnswer.schema.json

```
{
  "additionalProperties": false,
  "properties": {
    "assignment": {
      "minimum": 1,
      "maximum": 4,
      "type": "integer"
    },
    "due": {"type": "string"},
    "requirements": {
      "items": {"type": "string"},
      "type": "array"
    },
    "source": {"type": "string"}
  },
  "required": [
    "assignment", "due", "requirements", "source"
  ]
}
```

1 **Object boundary**

extra="forbid" becomes additionalProperties: false.

2 **Domain metadata**

Field limits become minimum and maximum.

3 **Required fields**

Every required field is named in the API schema.

Validation checks fit, not truth.

UNTRUSTED MODEL OUTPUT

`raw: JSON text`

→

HOST APPLICATION

`AssignmentAnswer.model_validate_json(raw)`

REJECTED

`assignment = 0`

Range violation

REJECTED

`delete = true`

Unexpected field

ACCEPTED SHAPE

`due = "tomorrow"`

String type fits

→

SEPARATE CHECK

Does the syllabus agree?

Validate the structure. Verify the facts against an authoritative source.

Validation is the last stop before side effects.

tool_boundary.py

```
01 try:
02     query =
AssignmentQuery.model_validate(raw_args)
03 except ValidationError as exc:
04     return invalid_input(exc)
05
06 n = query.assignment
07 return lookup_assignment(n)
```

1 **Validate raw arguments**

The model's payload is untrusted host input.

2 **Failure stops here**

ValidationError returns safe error data. The tool never runs.

3 **Run the tool**

Only query.assignment reaches lookup_assignment().

TRUST BOUNDARY

The model requests. Host code validates, authorizes, and executes.

Success and failure both need stable shapes.

tool_results.py

```python
class AssignmentResult(BaseModel):
    assignment: int
    due: str
    requirements: list[str]
    source: str

class ToolError(BaseModel):
    code: Literal[
        "invalid_input", "tool_failure"
    ]
    message: str
```

1 Grounded success

The source field preserves provenance downstream.

2 Branchable error

Literal limits the error codes the loop must handle.

3 Safe message

Return a concise explanation, never secrets or stack traces.

```
{"assignment": 1, "source": "syllabus"}
```

```
{"code": "tool_failure",
 "message": "Lookup failed"}
```

The host decides when to retry.

MODEL

Proposes

`assignment = 0`

1

2

PYDANTIC

Rejects

must be 1–4

HOST

Distills

concise error

3

4

POLICY

Decides

retry or stop

MODEL

Repairs

`assignment = 1`

5

BOUNDED RETRY BUDGET

Pydantic detects and explains. Host policy repairs, retries, or stops.